ĐẠI HỌC QUỐC GIA TPHCM

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN

KHOA CÔNG NGHỆ THÔNG TIN

BỘ MÔN CÔNG NGHỆ TRI THỨC

BÁO CÁO ĐỒ ÁN 02

Đề tài: CÁC HỆ MÃ DỰA TRÊN BÀI TOÁN LOGARITHM RỜI RẠC

Môn học: Nhập môn mã hóa – mật mã - 23CNTThuc2

Sinh viên thực hiện:

Lưu Vĩnh Phát - 23127448

Giáo viên hướng dẫn:

Thầy Nguyễn Đình Thúc

Thầy Nguyễn Văn Quang Huy

Thầy Trịnh Văn Minh

Ngày 7 tháng 12 năm 2025



Mục lục

1 Xử lý số lớn BigInt **1**

1.1 Biểu diễn số lớn và chuẩn hoá . . . 1
1.2 Phép cộng (add) và cộng số nhỏ (addSmall) . . . 2
1.3 Phép trừ (sub) . . . 2
1.4 Phép nhân số lớn (mul, mul64) . . . 3
1.5 Dịch phải 1 bit (shr1) . . . 3
1.6 Phép chia và lấy dư (divmod, divSmall) . . . 4
1.7 Các phép toán modulo: addMod, doubleMod, mulMod . . . 4
1.8 Lũy thừa modulo (powMod) . . . 5
1.9 Chuyển từ chuỗi hexa và kiểm tra nguyên tố . . . 6

2 Kiểm tra căn nguyên thủy modulo p **6**

2.1 Mô tả bài toán . . . 6
2.2 Định dạng dữ liệu vào/ra . . . 7
2.3 Ý tưởng thuật toán . . . 7
2.3.1 Tiêu chuẩn sử dụng . . . 7
2.3.2 Tính toán trên số lớn . . . 8
2.4 Luồng xử lý của chương trình `main` . . . 8

3 Hệ trao đổi khóa Diffie–Hellman **9**

3.1 Mô tả giao thức . . . 9
3.2 Định dạng dữ liệu vào/ra . . . 10
3.3 Chuyển đổi `Big` ↔ hex little-endian . . . 11
3.3.1 Từ hex little-endian sang Big (`parse_hex_le`) . . . 11
3.3.2 Từ Big sang hex big-endian (`to_hex_be`) . . . 11
3.3.3 Từ Big sang hex little-endian (`to_hex_le`) . . . 12
3.4 Thuật toán Diffie–Hellman trên BigInt . . . 12
3.4.1 Tính A, B, K . . . 12
3.4.2 Ví dụ đối chiếu với test mẫu . . . 13
3.5 Luồng xử lý trong `main` . . . 14

4 Hệ mã hóa ElGamal **14**

4.1 Mô tả hệ ElGamal . . . 14

4.1.1 Sinh khóa . . . 14

4.1.2 Mã hóa . . . 15

4.1.3 Giải mã . . . 15

4.2 Định dạng dữ liệu vào/ra . . . 15

4.3 Tính nghịch đảo modulo bằng định lý Fermat . . . 16

4.3.1 Cơ sở lý thuyết . . . 16

4.3.2 Cài đặt trên BigInt . . . 17

4.4 Thuật toán giải mã ElGamal trong chương trình . . . 17

4.4.1 Các bước chính . . . 17

4.4.2 Ví dụ minh hoạ theo test mẫu . . . 18

4.5 Luồng xử lý của chương trình `main` . . . 18

5 Hệ chữ ký ElGamal **19**

5.1 Mô tả hệ chữ ký ElGamal . . . 19

5.1.1 Sinh khóa . . . 19

5.1.2 Ký một thông điệp . . . 19

5.1.3 Xác thực chữ ký . . . 20

5.2 Định dạng dữ liệu vào/ra . . . 21

5.3 Thuật toán xác thực trên BigInt . . . 22

5.4 Ví dụ minh họa theo test mẫu . . . 23

5.5 Luồng xử lý của chương trình . . . 24

Tài liệu tham khảo **25**

Danh sách hình vẽ

1 Xử lý số lớn BigInt

Trong phần này, mô tả cách cài đặt và sử dụng cấu trúc `Big` để xử lý số nguyên rất lớn (vượt quá `uint64_t`).

1.1 Biểu diễn số lớn và chuẩn hoá

- Mỗi số lớn được lưu trong `vector<uint64_t> v`.
- Cơ số là 2^{64}, dạng little-endian:

$$\text{value} = v[0] \cdot (2^{64})^0 + v[1] \cdot (2^{64})^1 + \dots$$

- Số 0 được biểu diễn bằng `v.empty()`.

Hàm `normalize()` xoá các limb 0 ở cuối mảng:

- Nếu `v.back() == 0` thì pop ra cho đến khi không còn limb 0.
- Giúp kích thước luôn gọn, so sánh chính xác.

Một số hàm kiểm tra nhanh:

- `isZero()`: kiểm tra `v.empty()`.
- `isEven()`: kiểm tra bit 0 của `v[0]`.
- `cmp(a,b)`: so sánh hai Big (so chiều dài, rồi so từ limb cao xuống).
- `cmpSmall(x)`: so sánh với `uint64_t`.

Ví dụ:

```
// 2^64: v = [0, 1]
Big a;
a.v = {0ULL, 1ULL};

Big b(0);  // v = {}
bool z = b.isZero(); // true
```

1.2 Phép cộng (add) và cộng số nhỏ (addSmall)

Phép cộng hai BigInt. `Big::add(a,b)` cộng từng limb với carry, tương tự cộng bằng tay chỉ khác đây là trong cơ số 2^{64}:

- Duyệt i từ 0 tới `max(a.size, b.size)`.
- `sum = a[i] + b[i] + carry` (thiếu thì coi là 0).
- Cập nhật `carry` nếu tràn ra khỏi 64 bit.

Ví dụ:

```
Big a( (uint64_t)0xFFFFFFFFFFFFFFFFULL ); // 2^64 - 1
Big b(1);                                 // 1
Big c = Big::add(a, b);                   // c = 2^64 -> v = [0, 1]
```

Cộng số nhỏ (0..$2^{32}-1$). `addSmall(uint32_t d)` chỉ cộng vào limb thấp nhất và carry nếu cần.

Ví dụ:

```
Big x(100);
x.addSmall(25);  // x = 125
```

1.3 Phép trừ (sub)

`Big::sub(a,b)` giả sử `a >= b`:

- Duyệt từng limb, dùng `borrow` nếu `a[i] < b[i] + borrow`.
- Nếu cần mượn: cộng thêm 2^{64} rồi trừ.
- Cuối cùng `normalize()` để bỏ limb 0 dư.

Ví dụ:

```
Big a; a.v = {0ULL, 1ULL}; // 2^64
Big b(1);                  // 1
Big c = Big::sub(a, b);    // c = 2^64 - 1 -> v = [0xFFFFFFFFFFFFFFFF]
```

1.4 Phép nhân số lớn (mul, mul64)

Nhân hai số 64-bit. `mul64(a,b,low,high)`:

- Tách mỗi số thành 2 phần 32-bit: `a_hi, a_lo, b_hi, b_lo`.
- Nhân các cặp 32-bit và gom lại thành `low` (64 bit thấp) và `high` (64 bit cao).

Nhân hai BigInt. `Big::mul(a,b)`:

- Khởi tạo `r` với kích thước `a.size + b.size`.
- Với mỗi `i`, `j`: dùng `mul64(a.v[i], b.v[j], low, high)`.
- Cộng `low` vào `r.v[i+j]`, `high` vào `r.v[i+j+1]` với carry.

Ví dụ:

```
Big a(2);      // 2
Big b(3);      // 3
Big c = Big::mul(a, b); // c = 6

Big x( (uint64_t)0xFFFFFFFFFFFFFFFFULL );
Big y(2);
Big z = Big::mul(x, y); // z = 2^65 - 2
```

1.5 Dịch phải 1 bit (shr1)

`shr1()` chia số cho 2 (lấy floor):

- Duyệt từ limb cao xuống thấp.
- Mỗi limb: dịch phải 1 bit, ghép bit lẻ trước đó từ limb cao hơn.

Ví dụ:

```
Big a(10); // 1010_2
a.shr1();  // a = 5 (0101_2)
```

1.6 Phép chia và lấy dư (divmod, divSmall)

Chia Big cho Big: `divmod`. `Big::divmod(a,b,q,r)` tìm q, r sao cho:

$$a = q \cdot b + r, \quad 0 \le r < b$$

Ý tưởng:

- Duyệt limb của `a` từ cao xuống.
- Mỗi bước: `r = (r « 64) + a.v[i]`.
- Nếu `r < b` thì `q.v[i] = 0`; ngược lại dùng binary search để tìm `t` lớn nhất thoả `b * t <= r` rồi trừ.

Ví dụ:

```
Big a(1000);
Big b(37);
Big q, r;
Big::divmod(a, b, q, r);
// q = 27, r = 1  v   1000 = 27 * 37 + 1
```

Chia cho số nhỏ: `divSmall`. `divSmall(a, m)` với $1 \le m < 2^{32}$:

- Chia từ limb cao xuống, giữ phần dư trong `rem`.
- Cập nhật lại `a` thành thương, trả về `rem`.

Ví dụ:

```
Big a(1234567890ULL);
uint32_t r = divSmall(a, 1000);
// a = 1234567, r = 890
```

1.7 Các phép toán modulo: addMod, doubleMod, mulMod

Cộng modulo. `addMod(a,b,mod)`:

- Tính `s = add(a,b)`.
- Nếu `s >= mod` thì trừ `mod`.

Nhân đôi modulo. `doubleMod(a,mod)`:

- Tương tự `addMod(a,a,mod)`.

Nhân modulo (mulMod). `mulMod(a,b,mod)` dùng thuật toán *double-and-add*:

- `res = 0`.
- Trong khi `b != 0`:
  - Nếu `b` lẻ: `res = addMod(res, a, mod)`.
  - `a = doubleMod(a, mod)`.
  - `b.shr1()`.

Ví dụ:

```
Big a(13), b(19), mod(100);
Big c = Big::mulMod(a, b, mod); // c = 13*19 mod 100 = 47
```

1.8 Lũy thừa modulo (powMod)

`Big::powMod(base, exp, mod)` dùng *binary exponentiation*:

- `res = 1`.
- Trong khi `exp != 0`:
  - Nếu `exp` lẻ: `res = mulMod(res, base, mod)`.
  - `base = mulMod(base, base, mod)`.
  - `exp.shr1()`.

Ví dụ:

```
Big base(3), exp(13), mod(7);
Big r = Big::powMod(base, exp, mod); // r = 3^13 mod 7 = 3
```

1.9 Chuyển từ chuỗi hexa và kiểm tra nguyên tố

parse_hex_le. `parse_hex_le(s)`:

- Chuỗi `s` là hexa lưu dạng little-endian theo nibble.
- Hàm đảo chuỗi, sau đó duyệt từng kí tự: `val = val*16 + digit`.

Ví dụ:

```
Big x = parse_hex_le("0100000000000000");
```

Kiểm tra nguyên tố: isPrime. `isPrime(n)` dùng Miller–Rabin:

- Xử lý nhanh các trường hợp nhỏ (n $<$ 2, n $=$ 2, 3, n chẵn).
- Viết `n-1 = d * 2^s` với d.
- Dùng các cơ số cố định (2, 3, 5, ...) và powMod, mulMod để kiểm tra.

Ví dụ:

```
Big p(17);
Big q(18);
bool isP = isPrime(p); // true
bool isQ = isPrime(q); // false
```

2 Kiểm tra căn nguyên thủy modulo p

2.1 Mô tả bài toán

Cho p là một số nguyên tố lẻ (kích thước tới khoảng 512 bit), $U(p)$ là tập các ước nguyên tố dương của $p-1$, và g là một số nguyên dương. Ta cần kiểm tra:

g có phải là căn nguyên thủy modulo p hay không?

Theo lý thuyết, g là căn nguyên thủy modulo p khi và chỉ khi với mọi $q \in U(p)$ ta có:

$$g^{\frac{p-1}{q}} \not\equiv 1 \pmod{p}.$$

Do p có thể rất lớn, toàn bộ tính toán đều thực hiện trên cấu trúc số lớn `Big` (base 2^{64}) đã xây dựng: `parse_hex_le`, `powMod`, `divmod`, `dec1`, ...

2.2 Định dạng dữ liệu vào/ra

File đầu vào

File vào (ví dụ `test.inp`) gồm 4 dòng, tất cả đều viết ở dạng hexa **little-endian** (nibble thấp nhất đứng trước), dùng các ký tự `0-9, A-F`:

- Dòng 1: số nguyên tố lẻ p.
- Dòng 2: số lượng phần tử n của $U(p)$.
- Dòng 3: n số là các ước nguyên tố dương của $p-1$ (tập $U(p)$), cách nhau bởi khoảng trắng.
- Dòng 4: số g cần kiểm tra.

Mỗi chuỗi hexa này được đọc vào `std::string` rồi chuyển sang `Big` bằng hàm `parse_hex_le`.

File đầu ra

File ra (ví dụ `test.out`) chứa đúng **một dòng**:

- In `"1"` nếu g là căn nguyên thủy modulo p.
- In `"0"` nếu không.

Vì chỉ dùng số 0 và 1 nên việc ghi ở dạng hexa hay thập phân là như nhau.

2.3 Ý tưởng thuật toán

2.3.1 Tiêu chuẩn sử dụng

Gọi $\varphi = p-1$. Với mỗi ước nguyên tố $q \in U(p)$:

1. Kiểm tra q có thực sự chia hết φ hay không: nếu $q \nmid \varphi$ thì dữ liệu $U(p)$ không đúng, xem như test không hợp lệ $\Rightarrow$ trả về `0`.

2. Tính lũy thừa:

$$v_q = g^{\varphi/q} \bmod p.$$

3. Nếu tồn tại q sao cho $v_q \equiv 1 \pmod{p}$ thì g không phải căn nguyên thủy.

Nếu vượt qua toàn bộ các $q \in U(p)$ mà không rơi vào hai trường hợp loại trên, ta kết luận g là căn nguyên thủy modulo p.

2.3.2 Tính toán trên số lớn

Các phép toán cần dùng đều đã có trong thư viện `Big`:

- `Big::dec1(p)` để tính $\varphi = p - 1$.
- `Big::divmod(phi, q, exp, rem)` để chia φ cho q và lấy thương `exp` $= \varphi/q$ (nếu `rem` khác 0 thì $q \nmid \varphi$).
- `Big::powMod(g, exp, p)` để tính $g^{\varphi/q} \bmod p$ bằng lũy thừa nhanh modulo, không làm tràn.
- `val.cmpSmall(1ULL)` để so kết quả với 1.

2.4 Luồng xử lý của chương trình `main`

Chương trình chính chỉ làm nhiệm vụ đọc/ghi file và gọi hàm kiểm tra:

1. Đọc tên file vào/ra từ `argv[1]`, `argv[2]`, mở `ifstream` và `ofstream`.
2. Đọc 4 dòng vào các chuỗi `sp`, `sn`, `sq`, `sg`.
3. Chuyển từng chuỗi sang `Big`:

```
Big p   = parse_hex_le(sp);
Big nBig = parse_hex_le(sn);
uint64_t n = nBig.v.empty() ? 0ULL : nBig.v[0];

```

4. Đọc tiếp n chuỗi hexa cho $U(p)$, mỗi chuỗi parse thành một `Big` rồi lưu vào `std::vector<Big> primeFactors`.

5. Đọc chuỗi biểu diễn g, parse thành `Big g`.

6. Gọi hàm:

```
bool primitive = isPrimitiveRoot(p, primeFactors, g);

```

7. Ghi ra file output:

```
fout << (primitive ? "1\n" : "0\n");

```

Nhờ tận dụng sẵn các phép toán trên số lớn (cộng, nhân, lũy thừa modulo, chia), chương trình có thể xử lý các test với p đến 512 bit trong thời gian cho phép, đồng thời tuân thủ đúng định dạng hexa little-endian mà đề bài yêu cầu.

3 Hệ trao đổi khóa Diffie–Hellman

3.1 Mô tả giao thức

Cho:

- $p \in \mathbb{P}$ là số nguyên tố lẻ,
- g là một căn nguyên thủy modulo p,
- a là bí mật của Alice, b là bí mật của Bob.

Giao thức trao đổi khóa Diffie–Hellman diễn ra như sau:

1. Alice chọn bí mật a và tính giá trị công khai:

$$A = g^a \pmod{p}.$$

2. Bob chọn bí mật b và tính giá trị công khai:

$$B = g^b \pmod{p}.$$

3. Alice nhận B và tính khóa chung:

$$K_A = B^a = g^{ba} \pmod{p}.$$

4. Bob nhận A và tính khóa chung:

$$K_B = A^b = g^{ab} \pmod{p}.$$

Do $ab = ba$ nên $K_A = K_B = K = g^{ab} \pmod{p}$. Khóa K chỉ phụ thuộc vào p, g, a, b và không thể suy ra dễ dàng chỉ từ A và B nếu p đủ lớn.

3.2 Định dạng dữ liệu vào/ra

File đầu vào

File `test.inp` (tên có thể khác) gồm 4 dòng, tất cả số đều:

- là số nguyên không âm,
- được ghi ở dạng thập lục phân (hex), dùng `0-9, A-F`,
- theo quy ước **little-endian theo nibble** (ký tự đầu là nibble thấp nhất).

Cụ thể:

- Dòng 1: số nguyên tố lẻ p.
- Dòng 2: số g là căn nguyên thủy modulo p.
- Dòng 3: số bí mật a của Alice.
- Dòng 4: số bí mật b của Bob.

Mỗi chuỗi hex này được đọc vào `std::string` rồi chuyển sang `Big` bằng hàm `parse_hex_le` (đã trình bày ở phần BigInt).

Đầu ra của chương trình

Chương trình in ra 3 dòng (tương ứng file `test.out` hoặc `stdout`):

- Dòng 1: $A = g^a \bmod p$, dạng hex little-endian in hoa.
- Dòng 2: $B = g^b \bmod p$, dạng hex little-endian in hoa.
- Dòng 3: $K = g^{ab} \bmod p$, dạng hex little-endian in hoa.

Để sinh đúng format đề bài, chương trình cần chuyển ngược `Big` sang chuỗi hex little-endian.

3.3 Chuyển đổi `Big` ↔ hex little-endian

3.3.1 Từ hex little-endian sang Big (`parse_hex_le`)

`parse_hex_le` đã được dùng thống nhất trong toàn bộ các bài:

- Nhận vào chuỗi hex little-endian `s`.
- Đảo chuỗi `s` để thành big-endian.
- Duyệt từng ký tự, nhân số hiện tại với 16 rồi cộng thêm digit mới.
- Kết quả là một số lớn `Big` theo base 2^{64}.

Ví dụ, với `"56"`:

- Little-endian: `"56"` ⇒ big-endian: `"65"`.
- Đây là $0x65 = 101$, đúng với ví dụ test mẫu.

3.3.2 Từ Big sang hex big-endian (`to_hex_be`)

Trước khi xuất ra little-endian, ta sinh chuỗi big-endian không có số 0 dư ở đầu:

- Bắt đầu từ limb cao nhất của `Big`.
- Với limb cao nhất: bỏ qua các digit hex 0 ở đầu, chỉ in đến digit khác 0 đầu tiên.
- Với các limb tiếp theo: luôn in đủ 16 digit hex (64 bit).

3.3.3 Từ Big sang hex little-endian (`to_hex_le`)

Đề yêu cầu mọi số trong input/output đều ở dạng hex little-endian. Sau khi có chuỗi big-endian, ta chỉ cần đảo ngược:

```
// Big -> hex little-endian string (uppercase)
string to_hex_le(const Big &x)
{
    string be = to_hex_be(x);
    reverse(be.begin(), be.end()); // big-endian -> little-endian
    return be;
}
```

Ví dụ, nếu $A = 9$:

- `to_hex_be(A)` trả về `"9"`.
- `to_hex_le(A)` trả về `"9"` (đảo vẫn là chính nó).

Còn nếu $A = 0x65$:

- `to_hex_be(A)` trả về `"65"`.
- `to_hex_le(A)` trả về `"56"`, khớp với ví dụ đề bài.

3.4 Thuật toán Diffie–Hellman trên BigInt

3.4.1 Tính A, B, K

Toàn bộ tính toán số học được thực hiện trên kiểu `Big`. Thuật toán:

1. Tính $A = g^a \bmod p$ bằng hàm:

$$A = \texttt{Big::powMod(g, a, p)}.$$

2. Tính $B = g^b \bmod p$:

$$B = \texttt{Big::powMod(g, b, p)}.$$

3. Tính khóa chung K:

$$K = A^b \bmod p = \texttt{Big::powMod(A, b, p)}.$$

(Có thể dùng $K =$ `powMod(B, a, p)` tương đương.)

Logic này được gói lại trong một hàm tiện ích:

```
// Diffie-Hellman key exchange on Big
void diffieHellman(const Big &p, const Big &g,
                   const Big &a, const Big &b,
                   Big &A, Big &B, Big &K)
{
    A = Big::powMod(g, a, p); // A = g^a mod p
    B = Big::powMod(g, b, p); // B = g^b mod p
    K = Big::powMod(A, b, p); // K = A^b mod p = g^(ab) mod p
}
```

3.4.2 Ví dụ đối chiếu với test mẫu

Xét test mẫu 01 trong đề:

- Dòng 1: `56` $\Rightarrow$ big-endian: `65` $= 101 = p$.
- Dòng 2: `9` $\Rightarrow g = 9$.
- Dòng 3: `D1` $\Rightarrow$ big-endian: `1D` $= 29 = a$.
- Dòng 4: `21` $\Rightarrow$ big-endian: `12` $= 18 = b$.

Chương trình tính:

$$A = 9^{29} \bmod 101 = 9, \quad B = 9^{18} \bmod 101 = 8, \quad K = 9^{29 \cdot 18} \bmod 101 = 85.$$

Khi xuất ngược ra hex little-endian:

- $A = 9 \Rightarrow$ `"9"`,
- $B = 8 \Rightarrow$ `"8"`,
- $K = 85 = 0x55 \Rightarrow$ `"55"`.

Kết quả khớp với file `test.out` mẫu.

3.5 Luồng xử lý trong `main`

Chương trình `main` thực hiện các bước:

1. Nhận tên file input (và/hoặc output) từ tham số dòng lệnh, mở `ifstream` / `ofstream`.
2. Đọc 4 chuỗi hex little-endian: p, g, a, b.
3. Dùng `parse_hex_le` để chuyển từng chuỗi sang `Big`.
4. Gọi `diffieHellman(p, g, a, b, A, B, K)` để tính các giá trị.
5. Dùng `to_hex_le` để chuyển `A, B, K` sang chuỗi hex little-endian.
6. Ghi 3 dòng kết quả ra file/output đúng format đề bài.

Với cấu trúc BigInt đã xây dựng (đặc biệt là `powMod`), chương trình có thể mô phỏng giao thức Diffie–Hellman cho các modun p đến 512 bit trong thời gian giới hạn.

4 Hệ mã hóa ElGamal

Trong phần này, chúng tôi mô tả cách mô phỏng một số bước của hệ mã hóa ElGamal trên các số lớn `Big`. Bài toán yêu cầu từ khóa bí mật $d = (p, g, x)$ và bản mã $c = (c_1, c_2)$ phải sinh được:

- $h = g^x \bmod p$ (thành phần thứ ba của khóa công khai $e = (p, g, h)$),
- thông điệp gốc m thỏa $c = (c_1, c_2)$ là bản mã ElGamal của m.

4.1 Mô tả hệ ElGamal

4.1.1 Sinh khóa

Hệ ElGamal làm việc trên một trường hữu hạn $\mathbb{Z}_p$ với p là số nguyên tố lẻ và g là căn nguyên thủy modulo p:

1. Chọn ngẫu nhiên $p \in \mathbb{P}$ là số nguyên tố lẻ và g là căn nguyên thủy modulo p.
2. Chọn bí mật $x \in \mathbb{Z}_p^*$, $0 < x < p$.

3. Tính

$$h = g^x \bmod p.$$

4. Khóa công khai: $e = (p, g, h)$, khóa bí mật: $d = (p, g, x)$.

4.1.2 Mã hóa

Với khóa công khai $e = (p, g, h)$, một thông điệp m (thỏa $0 < m < p$) được mã hóa:

1. Chọn ngẫu nhiên $y \in \mathbb{Z}_p^*$, $0 < y < p$.
2. Tính $s = h^y \bmod p = g^{xy} \bmod p$.
3. Tính $c_1 = g^y \bmod p$.
4. Tính $c_2 = m \cdot s \bmod p$.

Bản mã là cặp (c_1, c_2).

4.1.3 Giải mã

Với khóa bí mật $d = (p, g, x)$ và bản mã $c = (c_1, c_2)$:

1. Tính $s = c_1^x \bmod p$. Vì $c_1 = g^y$ nên $s = g^{xy} \bmod p$, trùng với s khi mã hóa.
2. Tìm phần tử nghịch đảo s^{-1} modulo p, thỏa:

$$s^{-1} \cdot s \equiv 1 \pmod{p}.$$

3. Khôi phục thông điệp:

$$m = c_2 \cdot s^{-1} \bmod p.$$

4.2 Định dạng dữ liệu vào/ra

File đầu vào

File vào gồm 5 dòng, tất cả các số:

- là số nguyên không âm,

- được ghi ở dạng thập lục phân in hoa (`0-9, A-F`),
- theo quy ước **little-endian theo nibble** (ký tự đầu tiên là nibble thấp nhất).

Cụ thể:

- Dòng 1: số nguyên tố lẻ p.
- Dòng 2: số g là căn nguyên thủy modulo p.
- Dòng 3: số x bí mật, với khóa bí mật $d = (p, g, x)$.
- Dòng 4: c_1 – phần thứ nhất của bản mã $c = (c_1, c_2)$.
- Dòng 5: c_2 – phần thứ hai của bản mã.

Mỗi chuỗi hex được đọc vào `std::string` và chuyển sang `Big` bằng hàm `parse_hex_le` (đã mô tả ở phần BigInt). Ví dụ, chuỗi `"16"` (little-endian) tương ứng với `"61"` (big-endian) và giá trị $0x61 = 97$.

File đầu ra

Chương trình phải xuất ra 2 dòng, cũng ở dạng hex little-endian:

- Dòng 1: $h = g^x \bmod p$, với $h < p$.
- Dòng 2: thông điệp $m < p$ thu được sau khi giải mã bản mã $c = (c_1, c_2)$.

Để thỏa định dạng này, sau khi tính toán trên `Big`, kết quả được chuyển ngược sang chuỗi hex little-endian bằng hàm `to_hex_le`. Hàm này:

- trước hết tạo chuỗi hex big-endian không có số 0 dư ở đầu (`to_hex_be`),
- sau đó đảo chuỗi để thu được dạng little-endian đúng yêu cầu đề bài.

4.3 Tính nghịch đảo modulo bằng định lý Fermat

4.3.1 Cơ sở lý thuyết

Vì p là số nguyên tố, nhóm các phần tử khác 0 trong $\mathbb{Z}_p$ tạo thành một nhóm nhân có bậc $p - 1$. Định lý Fermat nhỏ cho biết:

$$a^{p-1} \equiv 1 \pmod{p}, \quad \forall a \in \mathbb{Z}_p^*.$$

Do đó:

$$a^{p-2} \equiv a^{-1} \pmod{p}.$$

Tức là nghịch đảo modulo p của a có thể được tính bằng một lũy thừa modulo duy nhất.

4.3.2 Cài đặt trên BigInt

Trong mã nguồn, nghịch đảo modulo được tính bằng:

$$a^{-1} \equiv a^{p-2} \pmod{p},$$

sử dụng hàm lũy thừa nhanh `Big::powMod`. Trước hết, ta lần lượt tính `p - 1` và `p - 2` bằng `Big::dec1`, sau đó gọi `powMod`
Do `powMod` đã được hiện thực an toàn trên số lớn (dùng bình phương - nhân và `mulMod`), cách tính này hoạt động tốt cho các mô-đun p đến 512 bit như đề bài.

4.4 Thuật toán giải mã ElGamal trong chương trình

4.4.1 Các bước chính

Từ các giá trị đầu vào p, g, x, c_1, c_2, chương trình thực hiện:

1. **Sinh h:**

   $$h = g^x \bmod p.$$

   Đây chính là thành phần thứ ba của khóa công khai $e = (p, g, h)$ mà đề yêu cầu in ra.

2. **Tính khóa tạm s:**

   $$s = c_1^x \bmod p.$$

   Vì $c_1 = g^y$ trong quá trình mã hóa, nên $s = g^{xy} \bmod p$, trùng với giá trị s ở bước mã hóa. Đây là điểm then chốt đảm bảo tính đúng đắn của giải mã.

3. **Tính nghịch đảo s^{-1} modulo p:**

   $$s^{-1} \equiv s^{p-2} \bmod p,$$

bằng hàm `modInverseFermat(s, p)` ở trên.

4. **Khôi phục thông điệp m:**

$$m = c_2 \cdot s^{-1} \bmod p.$$

Phép nhân và lấy modulo này được thực hiện bằng hàm `Big::mulMod`.

Toàn bộ logic trên được đóng gói trong một hàm elgamalDecrypt

4.4.2 Ví dụ minh hoạ theo test mẫu

Xét test mẫu 01 trong đề:

- Dòng 1: `16` $\Rightarrow$ big-endian: `"61"` $= 0x61 = 97 = p$.
- Dòng 2: `F5` $\Rightarrow$ big-endian: `"5F"` $= 0x5F = 95 = g$.
- Dòng 3: `71` $\Rightarrow$ big-endian: `"17"` $= 0x17 = 23 = x$.
- Dòng 4: `A1` $\Rightarrow$ `"1A"` $= 0x1A = 26 = c_1$.
- Dòng 5: `D5` $\Rightarrow$ `"5D"` $= 0x5D = 93 = c_2$.

Khi đó:

$$h = g^x = 23^{26} \bmod 97 = 95,$$

$$s = c_1^x = 26^{26} \bmod 97 = 16,$$

$$s^{-1} = 16^{-1} \bmod 97 = 91,$$

$$m = c_2 s^{-1} = 93 \cdot 91 \bmod 97 = 95.$$

Cả h và m đều bằng $95 = 0x5F$, nên file output chứa hai dòng `"F5"` (little-endian), phù hợp với test mẫu.

4.5 Luồng xử lý của chương trình `main`

Ở mức cao, chương trình chính thực hiện:

1. Đọc từ file 5 chuỗi hex little-endian tương ứng p, g, x, c_1, c_2.

2. Sử dụng `parse_hex_le` để chuyển từng chuỗi sang `Big`.
3. Gọi hàm `elgamalDecrypt` để tính h và m trên số lớn.
4. Chuyển h và m sang chuỗi hex little-endian bằng `to_hex_le`.
5. Ghi lần lượt 2 dòng kết quả ra file output.

Nhờ kết hợp BigInt (base 2^{64}), lũy thừa nhanh modulo và định lý Fermat, triển khai trên vẫn đáp ứng được yêu cầu thời gian cho các modun p lên đến 512 bit trong bộ test của đề bài.

5 Hệ chữ ký ElGamal

Trong phần này, chúng tôi mô tả cách mô phỏng quá trình *xác thực* chữ ký ElGamal trên số lớn, sử dụng các phép toán BigInt (`Big::powMod`, `Big::mulMod`, `Big::dec1`, `Big::cmp`, ...) đã xây dựng trước đó.

5.1 Mô tả hệ chữ ký ElGamal

5.1.1 Sinh khóa

Hệ chữ ký ElGamal làm việc trên trường hữu hạn $\mathbb{Z}_p$ với p là số nguyên tố lẻ và g là căn nguyên thủy modulo p:

1. Sinh ngẫu nhiên $p \in \mathbb{P}$ là số nguyên tố lẻ và $g \in \mathbb{Z}^+$ là căn nguyên thủy modulo p.
2. Chọn bí mật $x \in \mathbb{Z}^+$ sao cho $x < p$.
3. Tính
$$y = g^x \pmod{p}.$$
4. Khóa công khai dùng để xác thực: $v = (p, g, y)$. Khóa bí mật dùng để ký: $s = (p, g, x)$.

5.1.2 Ký một thông điệp

Với khóa bí mật $s = (p, g, x)$, để ký thông điệp m (giả sử $0 < m < p - 1$):

1. Chọn ngẫu nhiên $k \in \mathbb{Z}^+$ sao cho k và $p-1$ nguyên tố cùng nhau:

$$\gcd(k, p-1) = 1.$$

2. Tính

$$r = g^k \pmod{p}.$$

3. Tìm nghịch đảo k^{-1} modulo $p-1$:

$$k \cdot k^{-1} \equiv 1 \pmod{p-1}.$$

4. Tính

$$h = (m - xr)\, k^{-1} \pmod{p-1}.$$

5. Chữ ký của m là cặp $c = (r, h)$.

Từ định nghĩa trên, ta có hệ thức

$$h = (m - xr)\, k^{-1} \pmod{p-1} \quad \Longrightarrow \quad m \equiv xr + hk \pmod{p-1}.$$

Do đó:

$$g^m \equiv g^{xr+hk} = g^{xr}\, g^{hk} = (g^x)^r\, (g^k)^h = y^r\, r^h \pmod{p}.$$

5.1.3 Xác thực chữ ký

Với khóa công khai $v = (p, g, y)$, thông điệp m và chữ ký $c = (r, h)$, bên nhận xác thực chữ ký theo hai điều kiện:

1. Kiểm tra điều kiện biên:

$$0 < r < p, \quad 0 < h < p-1.$$

2. Tính $g^m \bmod p$ và $y^r r^h \bmod p$, sau đó kiểm tra:

$$g^m \equiv y^r r^h \pmod{p}.$$

Nếu cả hai điều kiện đều thỏa, chữ ký được xem là *hợp lệ* ; ngược lại là *không hợp lệ*.

5.2 Định dạng dữ liệu vào/ra

File đầu vào

File vào gồm 6 dòng, tất cả các số đều:

- là số nguyên không âm,
- được viết bằng các chữ số thập lục phân in hoa (`0-9, A-F`),
- theo dạng **little-endian theo nibble** (ký tự đầu tiên là nibble thấp nhất).

Cụ thể:

- Dòng 1: số nguyên tố lẻ p.
- Dòng 2: số g là căn nguyên thủy modulo p.
- Dòng 3: số y sao cho (p, g, y) tạo thành khóa công khai $v = (p, g, y)$.
- Dòng 4: thông điệp m cần xác thực (thỏa $m < p - 1$).
- Dòng 5: r – phần thứ nhất của chữ ký $c = (r, h)$.
- Dòng 6: h – phần thứ hai của chữ ký.

Trong chương trình, mỗi chuỗi hexa được đọc bằng `std::string` và chuyển sang số lớn `Big` bằng hàm `parse_hex_le`, giống như các bài trước.

File đầu ra

File ra chỉ có **một dòng**:

- In `"1"` nếu chữ ký *xác thực thành công* (hợp lệ),
- In `"0"` nếu chữ ký *xác thực thất bại*.

Do chỉ dùng các giá trị 0 và 1, cách ghi này tự động tương thích với yêu cầu "thập lục phân little-endian" của đề (0 và 1 là như nhau ở hệ 10 và hệ 16).

5.3 Thuật toán xác thực trên BigInt

Trong hiện thực, toàn bộ tính toán được thực hiện trên kiểu `Big`. Ta ký hiệu các phép toán chính:

- `Big::dec1(p)`: tính $p - 1$ trên số lớn.
- `Big::powMod(a, e, p)`: tính $a^e \bmod p$ bằng lũy thừa nhanh.
- `Big::mulMod(a, b, p)`: tính $a \cdot b \bmod p$ không bị tràn.
- `Big::cmp(a,b)`: so sánh hai `Big`.
- `isZero()`: kiểm tra một `Big` có bằng 0 hay không.

Quy trình xác thực được cài đặt tương đương với các bước sau:

1. Tính $p - 1$ dưới dạng BigInt:

$$p_1 = p - 1 = \texttt{Big::dec1(p)}.$$

2. **Kiểm tra điều kiện biên**:
   - Nếu $r = 0$ hoặc $h = 0$ thì chữ ký không hợp lệ.
   - Nếu $r \geq p$ thì chữ ký không hợp lệ.
   - Nếu $h \geq p - 1$ (tức $h \geq p_1$) thì chữ ký không hợp lệ.

   Các kiểm tra này được thực hiện bằng `isZero()` và `Big::cmp`.

3. **Kiểm tra đẳng thức** $g^m \equiv y^r r^h \pmod{p}$:

   (a) Tính

$$g^m \bmod p \quad \Rightarrow \quad \texttt{gm = Big::powMod(g, m, p)}.$$

   (b) Tính

$$y^r \bmod p \quad \Rightarrow \quad \texttt{yr = Big::powMod(y, r, p)},$$

   và

$$r^h \bmod p \quad \Rightarrow \quad \texttt{rh = Big::powMod(r, h, p)}.$$

(c) Nhân hai giá trị phía phải và lấy modulo:

$$\texttt{rhs} = \texttt{Big::mulMod(yr, rh, p)} \approx y^r r^h \bmod p.$$

(d) So sánh `gm` và `rhs` bằng `Big::cmp`. Nếu chúng bằng nhau thì chữ ký hợp lệ, ngược lại là không hợp lệ.

Tóm tắt lại, logic kiểm tra chữ ký trong chương trình gồm đúng hai tầng:

(1) Kiểm tra miền giá trị của r, h;
(2) Kiểm tra đẳng thức $g^m \equiv y^r r^h \pmod{p}$.

5.4 Ví dụ minh họa theo test mẫu

Xét **Test 01** trong đề bài:

- **Input (little-endian):**

Dòng 1: 16, Dòng 2: 71, Dòng 3: $F5$,
Dòng 4: 24, Dòng 5: $A5$, Dòng 6: $A5$.

- Khi chuyển sang big-endian và giá trị thập phân (theo giải thích đề bài):

$$p = 0x61 = 97, \quad g = 0x17 = 23, \quad y = 0x5F = 95,$$

$$m = 0x42 = 66, \quad r = 0x5A = 90, \quad h = 0x5A = 90.$$

Ta kiểm tra:

$$g^m = 23^{66} \bmod 97 = 85.$$

Tiếp theo:

$$y^r = 95^{90} \bmod 97 = 47, \qquad r^h = 90^{90} \bmod 97 = 8.$$

Do đó:

$$y^r r^h \equiv 47 \cdot 8 = 376 \equiv 85 \pmod{97}.$$

Ta thấy:

$$g^m \equiv 85 \equiv y^r r^h \pmod{97},$$

và các điều kiện biên $0 < r < p$, $0 < h < p - 1$ cũng đều thỏa. Vì vậy chương trình in ra `"1"`, đúng với file `test.out`.

5.5 Luồng xử lý của chương trình

Cuối cùng, luồng xử lý tổng thể của `main` như sau:

1. Đọc từ dòng lệnh tên file input và output, mở `ifstream` / `ofstream`.
2. Đọc 6 chuỗi hex little-endian: p, g, y, m, r, h.
3. Mỗi chuỗi được chuyển thành `Big` bằng `parse_hex_le`.
4. Gọi hàm xác thực chữ ký (dựa trên các bước đã mô tả ở trên).
5. Nếu chữ ký hợp lệ, ghi `"1"` vào file output; nếu không, ghi `"0"`.

Nhờ sử dụng chung hạ tầng BigInt đã xây dựng cho các bài trước, chương trình có thể xác thực chữ ký ElGamal với các mô-đun p lên đến 512 bit, đồng thời vẫn tuân thủ đúng toàn bộ yêu cầu định dạng dữ liệu vào/ra và giới hạn thời gian mà đề bài đặt ra.

Tài liệu

[1] D. E. Knuth, *The Art of Computer Programming, Volume 2: Seminumerical Algorithms*, 3rd ed. Addison–Wesley, Reading, MA, 1997.

[2] W. Diffie and M. E. Hellman, "New directions in cryptography," *IEEE Transactions on Information Theory*, vol. IT-22, no. 6, pp. 644–654, Nov. 1976.

[3] T. ElGamal, "A public key cryptosystem and a signature scheme based on discrete logarithms," *IEEE Transactions on Information Theory*, vol. 31, no. 4, pp. 469–472, 1985.

[4] "Diffie–Hellman key exchange," *Wikipedia, the free encyclopedia*. https://en.wikipedia.org/wiki/Diffie%E2%80%93Hellman_key_exchange.

[5] *ElGamal Cryptosystem Tutorial*, Cryptography Academy. https://www.cryptographyacademy.com/elgamal/.